Soup Kernel Challenge

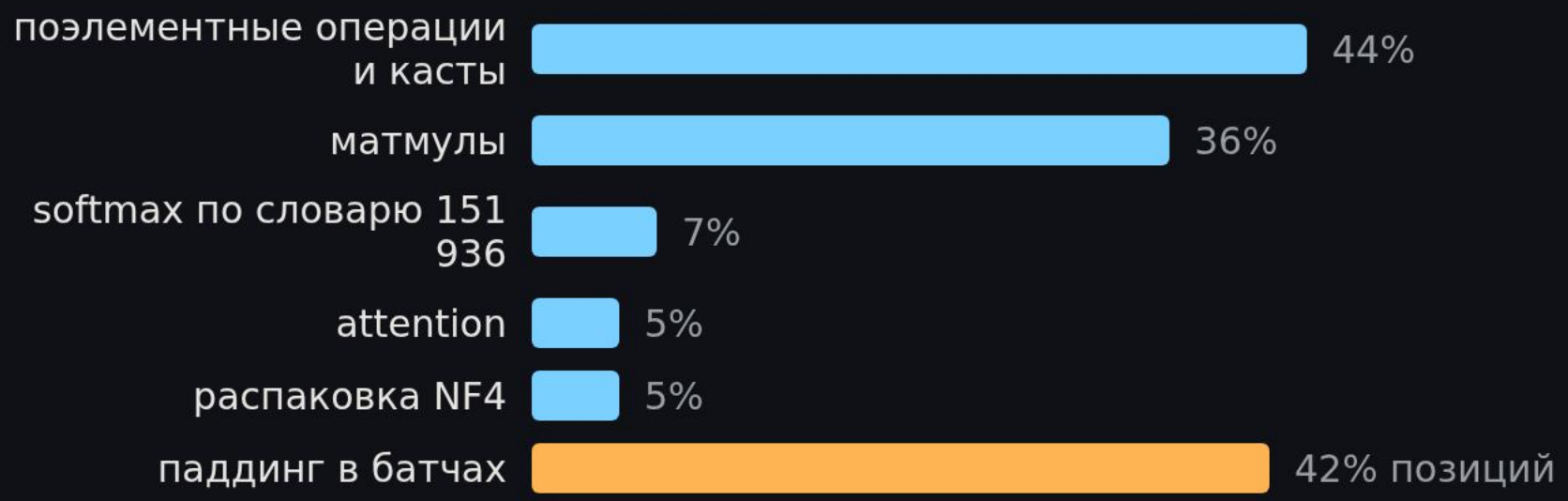
Triton-кернелы для soup train на Kaggle T4 · Студия Евгения Захарова

	База Soup 0.77.0	Решение	
Вся команда soup train (ноутбук жюри)	92.5 с	59.9 с	**1.54×**
Пик памяти GPU	14 345 МиБ	3 273 МиБ	**4.38×**
Eval loss адаптера	2.04963	2.04970	+0.003%

Qwen2.5-1.5B-Instruct · NF4 · LoRA r=16 · batch 4 × 512 · 64 шагов · один Tesla T4

Скрипты организаторов, протокол жюри: два прогона на сторону, все 10 проверок пройдены.

1 Куда уходил шаг базы на T4



- bitsandbytes после каждого 4-битного матмула кастит результат обратно в fp32
- логиты 4 × 512 × 151 936: fp16, fp32-копия, log-softmax, ещё два прохода TRL и Soup
- батч добивается до самой длинной строки: средняя строка 207 токенов, максимум 512

2 Слитый выходной слой и cross-entropy по кускам

- матмул головы по 256 позиций, логиты куска живут только внутри куска
- один кернел за один проход: log-sum-exp, потеря, градиент логитов, энтропия, argmax
- градиент по hidden считается тут же: ни одного полного тензора логитов в памяти
- колонки entropy, accuracy, num_tokens и построчные потери rewind — те же формулы

```
    row = tl.program_id(0)
    label = tl.load(labels_ptr + row)
    base = logits_ptr + row * stride_l
    m = -float("inf")
    s = 0.0
    best = -float("inf")
    best_idx = 0
    for start in range(0, V, BLOCK_V):
        cols = start + tl.arange(0, BLOCK_V)
        mask = cols < V
        l = tl.load(base + cols, mask=mask, other=-float("inf")).to(tl.f
        bm = tl.max(l, axis=0)
        m_new = tl.maximum(m, bm)
        s = s * tl.exp(m - m_new) + tl.sum(tl.exp(l - m_new), axis=0)
        m = m_new
        # the first index of the block maximum, kept only if it beats th
        is_max = l == bm
        cand = tl.min(tl.where(is_max, cols, V), axis=0)
        best_idx = tl.where(bm > best, cand, best_idx)
        best = tl.maximum(best, bm)
    lse = m + tl.log(s)
    labelled = label != IGNORE
```

3 Распаковка NF4 в один проход и склеенные веса

- bitsandbytes: два кернела на проекцию (absmax, веса) + cuBLAS, и так же в backward
- наш кернел: оба уровня квантования сразу, одна слитная запись fp16-строк
- q | k | v → одна матрица 2048 × 1536, gate | up → одна 17 920 × 1536
- одна распаковка и один GEMM вместо трёх узких; исходники освобождают память

```
    pid = tl.program_id(0)
    offs = pid * BYTES + tl.arange(0, BYTES)
    mask = offs < n_bytes
    byte = tl.load(packed_ptr + offs, mask=mask, other=0).to(tl.int32)
    hi = byte >> 4
    lo = byte & 15
    # value index i = 2 * byte index (hi) and 2 * byte index + 1 (lo); i
    block = (offs * 2) // BLOCK
    block2 = block // BLOCK2
    code = tl.load(absmax_u8_ptr + block, mask=mask, other=0).to(tl.int3
    scale2 = tl.load(scale2_ptr + block2, mask=mask, other=0.0)
    offset = tl.load(offset_ptr + block2, mask=mask, other=0.0)
    absmax = tl.load(code_ptr + code, mask=mask, other=0.0) * scale2 + o
    v_hi = tl.load(nf4_ptr + hi, mask=mask, other=0.0) * absmax
    v_lo = tl.load(nf4_ptr + lo, mask=mask, other=0.0) * absmax
    # interleave (hi, lo) pairs into one contiguous run of 2 * BYTES val
    v = tl.reshape(tl.join(v_hi, v_lo), [BYTES * 2])
    out_offs = pid * BYTES * 2 + tl.arange(0, BYTES * 2)
```

4 SwiGLU, RMSNorm, RoPE, fp16 между матмулами

- двумерный SwiGLU поверх склеенного gate|up-буфера, backward пишет один [R, 2I]-градиент
- RMSNorm: fp32-статистика, fp16 на выходе, backward из входа и одного скаляра на строку
- RoPE для q и k одним кернелом в каждую сторону вместо цепочки из десяти операций
- активации остаются в fp16, residual-поток fp32; таблица эмбеддингов снова в fp16, в котором Soup её загрузил
- кернелы компилируются в фоновом потоке, пока Soup грузит модель: первый шаг 0.9 с вместо 6 с
- gate|up последних 14 слоёв пересчитываются в backward: минус 0,7 ГБ, время не растёт

```
def rmsnorm_fwd_kernel(x_ptr, w_ptr, y_ptr, rstd_ptr, stride_x, stride_y
                       BLOCK_D: tl.constexpr):
    row = tl.program_id(0)
    cols = tl.arange(0, BLOCK_D)
    mask = cols < D
    x = tl.load(x_ptr + row * stride_x + cols, mask=mask, other=0.0).to(
    var = tl.sum(x * x, axis=0) / D
    rstd = 1.0 / tl.sqrt(var + eps)
    tl.store(rstd_ptr + row, rstd)
    w = tl.load(w_ptr + cols, mask=mask, other=0.0).to(tl.float32)
    y = w * (x * rstd)
    tl.store(y_ptr + row * stride_y + cols, y.to(y_ptr.dtype.element_ty)
```

5 Паддинг убран внутри батча

- после эмбеддингов остаются только реальные токены: [1, N_real, D]
- нормы, проекции, LoRA, MLP и выходной слой считают по ним
- attention: причинный SDPA по каждой строке на её токенах, без маски и блоков паддинга
- строки между батчами не смешиваются; маски dropout у LoRA те же, что у базы

```
def per_row_attention(q, k, v, lengths, head_dim, groups, scaling, dropo
    """Causal SDPA of each sequence on its own rows. q, k, v: [1, N_real
    row-major by sequence; `lengths` the sequence lengths in order. Retu
    outs, start = [], 0
    q2, k2, v2 = q[0], k[0], v[0]
    for L in lengths:
        if L == 0:
            continue
        stop = start + L
        qi = q2[start:stop].view(1, L, -1, head_dim).transpose(1, 2)
        ki = k2[start:stop].view(1, L, -1, head_dim).transpose(1, 2)
        vi = v2[start:stop].view(1, L, -1, head_dim).transpose(1, 2)
        if groups > 1:  # the same as transformers' repeat_kv for groupe
            ki = ki.repeat_interleave(groups, dim=1)
            vi = vi.repeat_interleave(groups, dim=1)
        oi = torch.nn.functional.scaled_dot_product_attention(qi, ki, vi
                                                              is_causal=
        outs.append(oi.transpose(1, 2).reshape(L, -1))
        start = stop
    return torch.cat(outs, dim=0).unsqueeze(0)
```

6 Замер по протоколу жюри

	База	Решение	
soup train, ноутбук жюри	92.5 с	59.9 с	**1.54×**
пик памяти, nvidia-smi	14 345 МиБ	3 273 МиБ	**4.38×**
eval loss	2.04963	2.04970	+0.003%

Кривая потерь: первый шаг 0.00%, в среднем 0.01%. Два прогона на сторону в другой сессии: 1.49× / 4.41×. Стриминг весов и пересчёт gate|up в 14 последних слоях бесплатны по времени: шаг 0.42 с.

- ✓ все прогоны завершились
- ✓ свои @triton.jit работают в обучении
- ✓ та же задача и выходы Soup
- ✓ качество: eval loss в допуске
- ✓ чистый замер на T4
- ✓ без готовых кернел-библиотек
- ✓ доля своих кернелов ≥ 5%
- ✓ кривая потерь как у базы
- ✓ не хуже базы по времени и памяти
- ✓ 79 регрессионных тестов Soup

7 Ограничения и что дальше

- attention остаётся на SDPA PyTorch, по строкам; свой кернел по упакованным строкам — следующий шаг
- распаковка NF4 отдельно от GEMM: слитый вариант должен обогнать cuBLAS на T4
- патчи рассчитаны на Qwen2 + NF4 (блоки 64, двойное квантование) + обычную LoRA PEFT
- в репозитории: код, тест каждого кернела против PyTorch и bitsandbytes, сырые результаты всех прогонов

github.com/BAITC-Hacks/hack-a8ca734b-solo-ai-team